



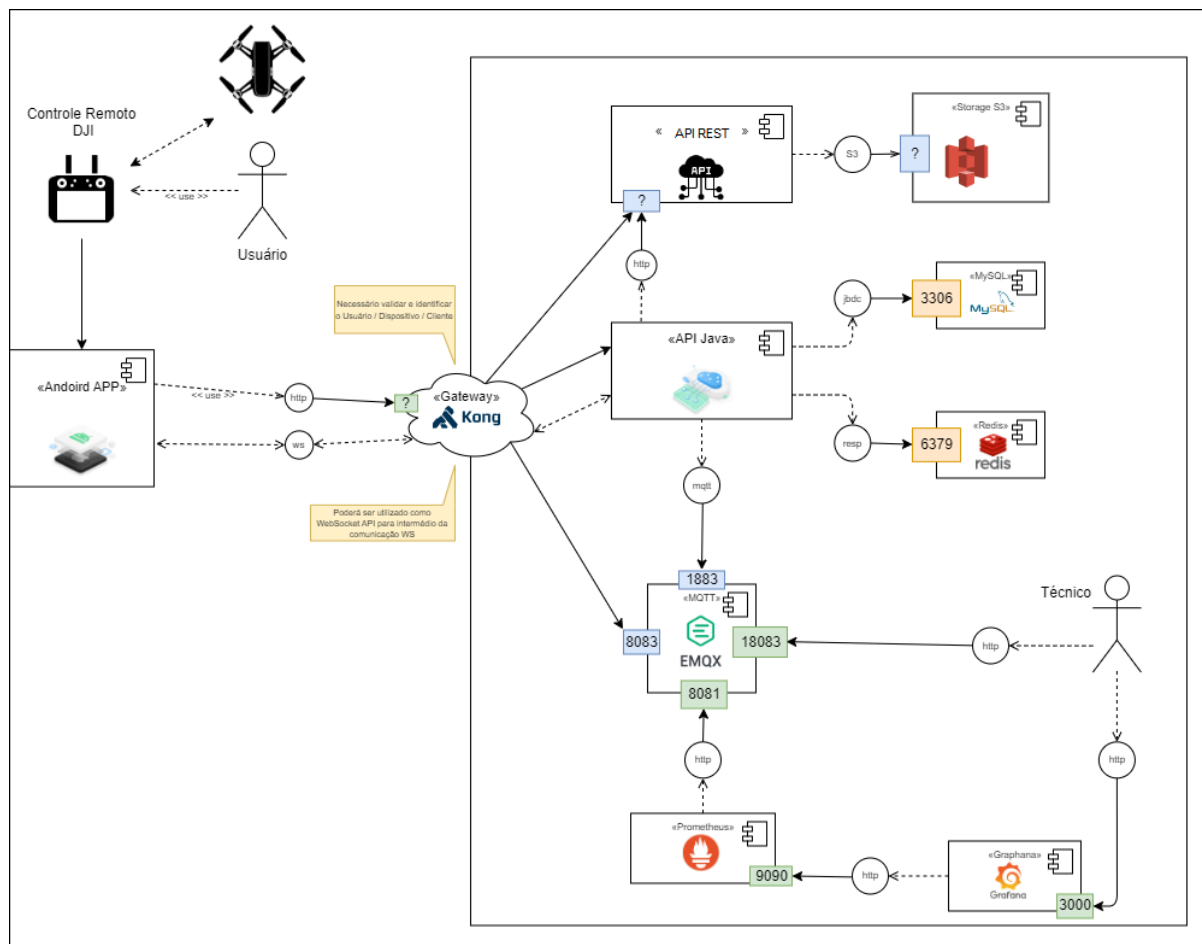
Inteligência Artificial: Back-end

Prova Prática

Instruções

- A prova é composta por 6 partes;
- Responda todas as questões com o máximo de detalhes possíveis, ou seja, justifique suas decisões técnicas, não só "o que" fez, mas "por quê";
- Stack esperada: **Python, FastAPI, Pydantic, PostgreSQL, Redis, Docker, AWS, React/Vite (front, se aplicável)**;
- O código prático deve ficar em um **repositório Git** (GitHub ou GitLab, público ou com acesso liberado para nós), com commits organizados.
- É permitido usar IA (Copilot, ChatGPT, Claude etc.) para apoiar o desenvolvimento, **mas a prova deve refletir o seu raciocínio e trabalho**. No README, inclua uma seção "**Uso de IA**" descrevendo o que foi gerado/apoiado por IA e o que você escreveu/revisou manualmente;
- Você tem **72h (3 dias)** para entregar. Se não conseguir concluir algum item, descreva por escrito como faria com mais tempo! Isso também é avaliado. Entendemos que sua vida pode estar corrida.
- Documente e comente o código onde agregar clareza, sem exagerar em comentários óbvios.
- Qualidade é um pilar dos nossos produtos, portanto, **sinta-se à vontade para implementar testes unitários nos pontos que considerar necessário**.

Parte 1: Questão teórica



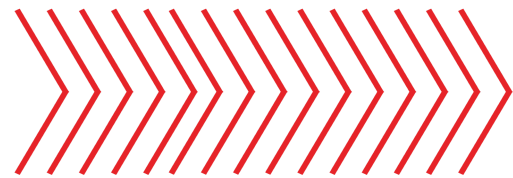
O diagrama acima apresenta uma estrutura quase idêntica à estrutura base da DJI Cloud API que é utilizada para disponibilizar os serviços de drone da DJI por outras empresas. Entre as adaptações temos a implementação de um microserviço abstraindo a comunicação com a Storage S3 para manter uma maior segurança nos dados, também foi inserido um Proxy Reverso (Gateway - Kong) para disponibilização de todos os serviços. E para fins de monitoramento, foram implementados serviços como o Prometheus e o Grafana. Com base no diagrama acima, descreva de maneira geral o objetivo de cada componente.

Parte 2: Desenvolvimento de API RESTful

Questão 1:

Crie uma API RESTful utilizando Flask ou Fast API que permita a criação, leitura, atualização e exclusão (CRUD) de registros de missões. Cada missão deve ter os seguintes campos:

- id
- name
- status



- created_at
- drone_model
- image_count
- area_hectares

Requisitos:

- Implemente todas as rotas necessárias para realizar as operações CRUD.
- Utilize um banco de dados SQLite para armazenar os dados das missões.
- Garanta que a API seja segura, incluindo autenticação básica (ex.: token JWT).

Parte 3: Integração de Modelos de IA

Questão 2:

A empresa utiliza modelos de IA para processar imagens aéreas. Implemente uma API que receba uma solicitação de processamento, valide os parâmetros recebidos, execute a inferência utilizando um modelo previamente treinado e registre o histórico da execução.

Requisitos:

- modelo carregado apenas uma vez
- tratamento de erro
- tempo de inferência
- versionamento do modelo
- histórico das previsões

Parte 4: Docker e Orquestração de Contêineres

Questão 3:

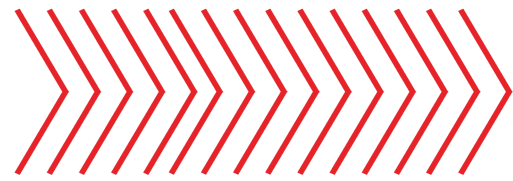
Sua API será utilizada em um ambiente de processamento de imagens de drones, onde múltiplos usuários poderão enviar solicitações simultaneamente.

Você deverá:

- Containerizar a aplicação utilizando Docker;
- Criar um docker-compose contendo:
 - API
 - PostgreSQL
 - Redis
- Configurar variáveis de ambiente;
- Implementar um healthcheck da API;
- Garantir que a aplicação aguarde a disponibilidade do banco antes de iniciar.

Além da implementação, explique:

- Por que utilizar Redis neste cenário?



- Como escalaria a aplicação para processar milhares de imagens simultaneamente?
- Como faria o deploy em AWS?
- Como desacoplaria o processamento pesado da API?

Parte 5: Questões extras

Questão 4. Um usuário envia 500 imagens de drone. O processamento pode levar vários minutos. Descreva uma arquitetura para esse fluxo.

Questão 5. O upload de uma imagem de 2 GB não deve passar pela API. Como você resolveria isso?

Questão 6. Como impedir que um usuário baixe imagens pertencentes a outro cliente?

Parte 6: Portfólio

Questão 7. Envie link(s) do seu GitHub/GitLab com projetos anteriores ou pessoais relevantes (se houver). Escolha **um projeto** que considere mais representativo do seu nível técnico e explique brevemente:

- Qual problema ele resolve;
- Principais decisões técnicas e por quê;
- O que você faria diferente hoje, com o que aprendeu desde então.

Envio da Prova

- Link do repositório Git com todo o código-fonte, Dockerfile e docker-compose.yml;
- README com: instruções de execução, respostas das questões teóricas, seção "Uso de IA", e link(s) de portfólio (Parte 5);
- Nomeie o repositório como: `prova-backend-c-[SeuNome]`;
- Envie o link do repositório para o endereço de e-mail fornecido.

Boa sorte! 🍀